

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação



UNIVERSIDADE FEDERAL DE UBERLÂNDIA
FACULDADE DE MATEMÁTICA
2019

AUTOR(A) DA DISSERTAÇÃO

Título da Dissertação

Dissertação apresentada ao Programa de Pós-Graduação...

UBERLÂNDIA - MG
2010

foi quando a vida me deu um doce

Sumário

Introdução	5
1 Introdução a Redes Neurais	6
1.1 Perceptron	6
1.1.1 Neurônio Biológico	6
1.1.2 Neurônio Artificial	7
1.1.3 Tipos de Função de Ativação	7
1.1.4 Algoritmo de Aprendizagem do Perceptron	8
1.2 Redes Neurais	11
1.2.1 Arquiteturas de Redes Neurais	11
1.2.2 Backpropagation	13
1.2.3 Gradiente Descendente Estocástico	14
1.3 Overfitting	14
1.3.1 Hyperparâmetros	15
1.3.2 Regularização	15
1.3.3 Momentum	17
1.3.4 Vanishing Gradient	17
1.4 Redes Neurais Convolucionais	18
1.4.1 Cotex Visual Humano	19
1.4.2 Convolução	19
1.4.3 <i>Kernel</i> Multicamadas	21
1.4.4 Padding	21
1.4.5 Pooling	22
1.4.6 Stride	22
1.4.7 Convolução Transposta	23
1.5 U-net	23
1.5.1 Copy and Crop	24

Introdução

Helloo baby

Capítulo 1

Introdução a Redes Neurais

1.1 Perceptron

Colocar uma explicação para a seção

1.1.1 Neurônio Biológico

Para entender o funcionamento do neurônio matemático é necessário entender o funcionamento do neurônio biológico. Neurônios são “células excitáveis” que se comunicam ao transmitir impulsos elétricos. Células *excitáveis* são definidas como células capazes de produzir sinais elétricos amplos e rápidos denominados *potências de ação* (veja [8]). A maioria dos neurônios contém três componentes principais: um corpo celular, o(s) dendrito(s) e um axônio. O corpo celular, chamado de *soma* contém o núcleo celular, local onde parte da informação recebida de outras partes do corpo é manipulada. Os *dendritos* se ramificam a partir do corpo celular, são responsáveis por receber e integrar uma imensa quantidade de sinais provenientes de outros neurônios. Outra ramificação advinda do corpo celular é o *axônio*. Diferente dos dendritos, cuja função é *receber* informação, um axônio *envia* informações. Grande parte dos neurônios tem apenas um axônio, mas os axônios conseguem se ramificar, enviando, assim, sinais a mais de um destino. Esses três componentes, de forma resumida, formam o neurônio biológico, como mostrado na Figura 1.1.

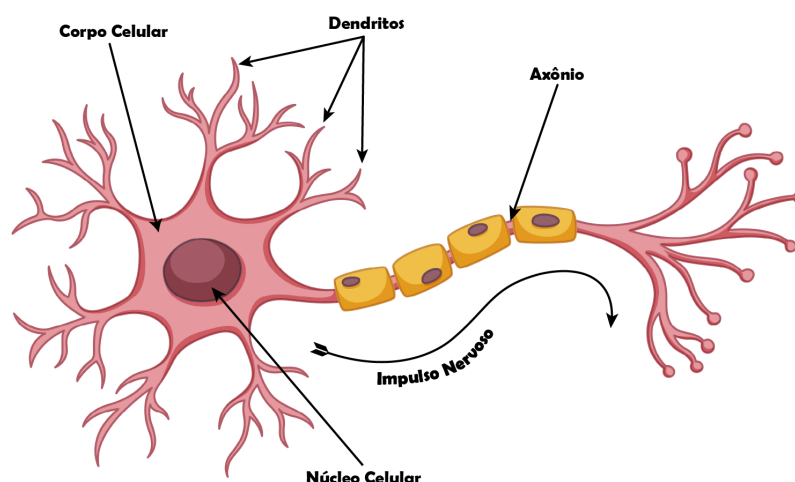


Figura 1.1: Estrutura de um neurônio típico.

Outro fator importante em um neurônio biológico são os potenciais de ação, gerados nos dendritos, são avaliados e ponderados por meio da *força sináptica* [1], que nos diz o quão importante a informação. Após esse processo, são enviados para o núcleo celular. Entre o corpo celular e o axônio será decidido se o potencial é excitatório ou inibitório. A decisão é feita caso o potencial de ação supere ou não um *limiar*. Caso seja excitatório, a informação é passada a diante, caso seja inibitório, a informação é suprimida.

1.1.2 Neurônio Artificial

A partir das informações obtidas com o estudo do neurônio biológico, pesquisadores tentaram simular seu funcionamento em computador. A implementação mais bem aceita de simulação foi proposta por dois pesquisadores, o neurobiologista Warren McCulloch e o matematico Walter Pitts em 1943 em [11] que utilizaram-se da lógica proposicional para descrever o funcionamento do sistema nervoso central 1.2. McCulloch acreditava que o neurônio bilógico tinha uma natureza binária, ou seja, um neurônio influencia outro ou não, entretanto, posteriormente essa crença mostrou-se errônea com os avanços de estudos na área.

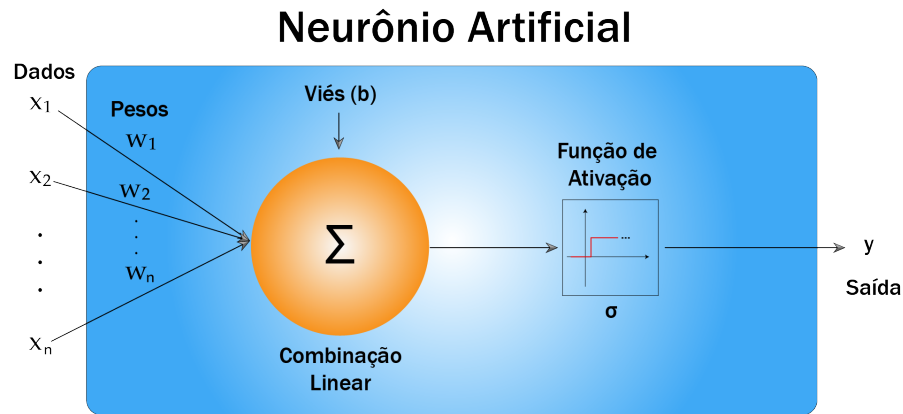


Figura 1.2: Modelo de um neurônio de McCulloch e Pitts.

As variáveis $x_1, x_2, x_3, \dots, x_n$ são denominadas *dados de entrada*, em analogia às sinapses de um neurônio biológico. Os valores $w_1, w_2, w_3, \dots, w_n$ são os *pesos* associados a cada dado de entrada, os pesos nos dizem o quão importante é a entrada para a *soma*, parte central em laranja na Figura 1.2. O viés tem a mesma funcionalidade do limiar no neurônio biológico, determinando se o neurônio será ativado ou não. Para resumir a notação, defini-se o viés de b (do inglês, *bias*). Na soma, a variável z é obtida fazendo a combinação linear das variáveis $x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \dots + x_n * w_n + b = z = \sum_{i=1}^n w_i x_i + b$. Em seguida, o valor z é aplicado a uma *função de ativação*, resultando na saída $\hat{y}$, ou seja, $\hat{y} = \sigma(\sum_{i=1}^n w_i x_i + b)$, para $\sigma : \mathbb{R} \rightarrow \mathbb{R}$.

1.1.3 Tipos de Função de Ativação

A função de ativação, representada por σ , define a saída de um neurônio, pode-se definir diferentes tipos de funções de ativação, como a função de limiar, função linear por partes e a função sigmoidal.

- *Função de Limiar*: Para este tipo de função de ativação, descrito na Equação 1.1, o neurônio é ativado (ou seja, produz uma saída de 1) se a soma ponderada das entradas

mais o viés for maior ou igual a zero. Caso contrário, a saída é 0.

$$\sigma(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases} \quad (1.1)$$

- *Função Sigmoidal*: A função sigmoidal, cujo o gráfico tem a forma de um "S", é de longe a forma mais comum de função de ativação utilizada na construção de neurônios artificiais. Ela é definida como uma função estritamente crescente que exibe um balanceamento adequado entre comportamento linear e não-linear. Um exemplo de função sigmoidal é a *sigmoid logística*, definida na Equação 1.2:

$$\sigma(z) = \frac{1}{1 + \exp(-\alpha z)}, \quad (1.2)$$

onde α é um parâmetro que controla a inclinação da curva. A função sigmoid logística tem um conjunto de propriedades que se mostrarão muito úteis nos cálculos relacionados à aprendizagem dos pesos:

1. Não linear;
 2. Contínua e diferenciável em todo o domínio de $\mathbb{R}$;
 3. A derivada tem forma simples e é expressa pela própria função: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$;
 4. É estritamente monótona, ou seja, $z_1 \leq z_2 \iff \sigma(z_1) \leq \sigma(z_2)$.
- *Função Rectified Linear Unit (ReLU)*: A função ReLU é definida como $\sigma(z) = \max(0, z)$. Ela é amplamente utilizada em redes neurais profundas (estudada na seção 1.2) devido à sua simplicidade computacional e capacidade de mitigar o problema do *Vanish Gradient*. A função ReLU ativa o neurônio apenas quando a soma ponderada das entradas mais o viés é positiva, caso contrário, a saída é zero, ou seja:

$$\sigma(z) = \begin{cases} z, & \text{se } z > 0 \\ 0, & \text{se } z \leq 0 \end{cases} \quad (1.3)$$

Em geral, o objetivo da função de ativação é limitar a saída de um neurônio para um intervalo de valores, a Figura 1.3 (a) mostra o gráfico da função sigmoid logística, enquanto a Figura 1.3 (b) mostra o gráfico da função de limiar.

A principal desvantagem no neurônio de McCulloch e Pitts é a ausência de um algoritmo de aprendizagem, ou seja, uma maneira de ajustar os pesos e o viés do neurônio para que ele seja capaz de resolver um problema específico. Essa limitação foi superada em 1958 por Frank Rosenblatt, que propôs o *Perceptron* [6], um modelo de neurônio artificial com um algoritmo de aprendizagem capaz de ajustar os pesos e o viés do neurônio.

1.1.4 Algoritmo de Aprendizagem do Perceptron

O Perceptron é um modelo de neurônio artificial que utiliza um algoritmo de aprendizagem supervisionada para ajustar seus pesos e viés com base em um conjunto de dados "rotulados". Dados rotulados são conjuntos de dados de entrada associados a saídas desejadas, ou seja, para cada conjunto de entradas, há uma saída correta conhecida. O objetivo do Perceptron é encontrar uma função que mapeie corretamente as entradas para as saídas desejadas, minimizando o erro entre as previsões do modelo e os valores reais.

Para esclarecer o funcionamento do Perceptron e do algoritmo de aprendizagem, é proposto o Exemplo 1.1.

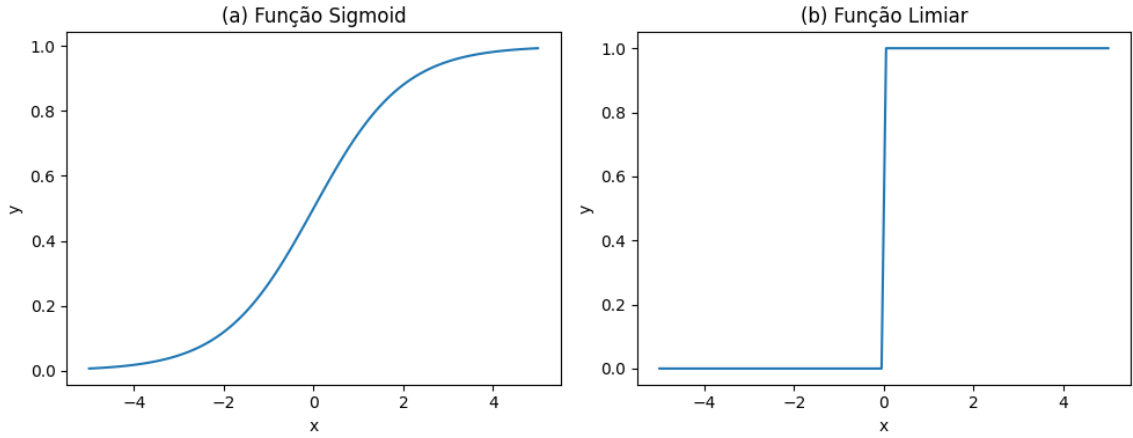


Figura 1.3: Funções de Ativação.

Exemplo 1.1 *Um investidor busca decidir qual ação adquirir com base nas variáveis: **é caro?**, **é confiável?** e **é volátil?**. Para isso, dispõe das avaliações de quatro colegas que também analisam essas mesmas características antes de realizar seus investimentos.*

Tabela 1.1: Banco de dados com as avaliações dos colegas e a decisão final de compra.

	valor	confiança	volatilidade	y
x_1	0	1	0	1
x_2	1	0	1	0
x_3	1	1	1	0
x_4	0	0	0	1

Em que $0 = \text{não}$, $1 = \text{sim}$ e $x_2, \dots, x_4$ representam as respostas de cada colegas e y a decisão final de cada um. Com isso, o Perceptron terá três variáveis de entrada e como saída, a decisão sobre a compra da ação. O Perceptron, portanto, possui três variáveis de entrada e uma saída que representa a decisão de compra. Os pesos iniciais são definidos como: $w_1 = w_2 = w_3$ e o viés como $b = 0$. A função de ativação utilizada é a **função limiar** (1.1). Testando o primeiro dado de entrada $x_1 = (0, 1, 0)$ e $y = 1$, obtém-se:

$$z = w \cdot x_1 + b = 0 \Rightarrow \hat{y} = \sigma(0) = 0$$

Como $y = 1 \neq \hat{y} = 0$, em que $\hat{y}$ é a resposta da rede, o Perceptron comete um erro e é necessário aplicar o **algoritmo de aprendizagem** (o neurônio de McCulloch e Pitts pararia aqui). Para isso, define-se uma função de custo, responsável por quantificar o erro cometido durante o treinamento. Neste exemplo, utiliza-se o **Erro Quadrático Médio**, dado por:

$$C(w, b) = \frac{\sum_{i=1}^k (y_i - \hat{y}_i)^2}{n},$$

em que y_i é a resposta do dado de entrada x_i e n representa a quantidade de dados presentes no conjunto de treinamento.

O processo de treinamento ajusta iterativamente os pesos e o viés por meio do método da descida do gradiente, cujo objetivo é minimizar a função de custo. Esse método utiliza as derivadas parciais da função de custo em relação aos pesos e ao viés, o que indica a direção de maior crescimento da função. Para reduzir o erro, atualizam-se os parâmetros na direção oposta ao gradiente. As derivadas são:

$$\frac{\partial C(w, b)}{\partial w_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z) \cdot x_i}{n}; \quad (1.4)$$

$$\frac{\partial C(w, b)}{\partial b_i} = -2 \cdot \frac{\sum_{i=1}^k (y_i - \hat{y}_i) \cdot \sigma'(z)}{n}. \quad (1.5)$$

No o Exemplo 1.1 assume-se $\sigma'(z) = 1$ para simplificação. Como a derivada aponta para a região de maior valor da função, multiplica-se por -1 para obter a direção da região de minimização. Com isso, é obtido o seguinte algoritmo:

Input: Taxa de aprendizado η , função de custo $C(w, b)$, pesos iniciais $w_1, w_2, \dots, w_n$ e viés b

Output: Pesos e viés ajustados

Inicializar os pesos w_i e o viés b com valores pequenos aleatórios;

repeat

 Calcular a saída do modelo;

$\hat{y}_i \leftarrow \sigma(\sum_{i=1}^n w_i x_i + b)$;

 Calcular o erro;

$\Delta_w \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z) \cdot x_i}{n}$;

$\Delta_b \leftarrow \frac{\sum_{i=1}^n (y_i - \hat{y}_i) \cdot \sigma'(z)}{n}$;

 Atualizar os pesos e o viés;

$w_i \leftarrow w_i + \eta \Delta_w$, para todo $i = 1, 2, \dots, n$;

$b \leftarrow b + \eta \Delta_b$;

until convergência ou número máximo de iterações;

return w_i, b ;

Algorithm 1: Método da Descida do Gradiente

A taxa de aprendizagem (η) presente no algoritmo (1) pode ser interpretada como o tamanho do passo dado em direção ao mínimo da função de custo. Quando o valor dessa taxa é muito pequeno, o processo de convergência torna-se mais lento, exigindo um número maior de iterações para atingir o mínimo. Por outro lado, se o passo for muito grande, o algoritmo pode ultrapassar o ponto de mínimo, dificultando ou até impedindo a convergência adequada, para o Exemplo 1.1 é utilizada uma taxa de aprendizado de 0.1.

Colocar a taxa de aprendizado

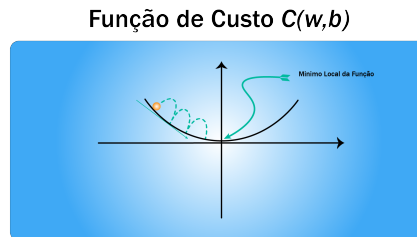


Figura 1.4: Representação da taxa de aprendizagem.

Aplicando o algoritmo 1 no Exemplo 1.1, os seguintes ajustes são feitos nos pesos e no viés após uma passada pelos 4 dados presentes no banco de dados:

$$w = (-0.1, 0.1, -0.1), \quad b = 0.1$$

Verificando as previsões da rede para os parâmetros atualizando, é obtido:

$$x_1: (0, 1, 0) \Rightarrow z = 0 \cdot (-0.1) + 1 \cdot 0.1 + 0 \cdot (-0.1) + 0.1 = 0.2 \Rightarrow \hat{y} = 1;$$

$$x_2: (1, 0, 1) \Rightarrow z = -0.1 + 0 + -0.1 + 0.1 = -0.1 \Rightarrow \hat{y} = 0;$$

$$x_3: (1, 1, 1) \Rightarrow z = -0.1 + 0.1 - 0.1 + 0.1 = 0 \Rightarrow \hat{y} = 0;$$

$$x_4: (0, 0, 0) \Rightarrow z = 0 + 0 + 0 + 0.1 = 0.1 \Rightarrow \hat{y} = 1.$$

Assim, após uma única época com $\eta = 0.1$ os pesos e o viés encontrados classificam corretamente todos os exemplos do conjunto de treinamento.

Comumente o banco de dados é dividido em dois conjuntos: o conjunto de treinamento e o conjunto de teste. O conjunto de treinamento é utilizado para ajustar os pesos e o viés do Perceptron, enquanto o conjunto de teste é utilizado para avaliar o desempenho do modelo em dados não vistos durante o treinamento. Essa divisão é essencial para garantir que o modelo aprenda a generalizar bem para novos dados, evitando o problema do *overfitting* (visto na seção 1.3), que ocorre quando o modelo se ajusta excessivamente aos dados de treinamento, perdendo a capacidade de generalização.

1.2 Redes Neurais

As redes neurais são um “encadeamento” de múltiplos neurônios artificiais, onde a saída de um neurônio é a entrada de outro. Redes neurais podem ser organizadas em camadas: uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. Cada camada é composta por vários neurônios, e cada neurônio em uma camada está conectado a todos os neurônios na camada seguinte. A Figura 1.5 ilustra a estrutura básica de um neurônio artificial em comparação com o funcionamento de uma rede neural com duas camadas ocultas.

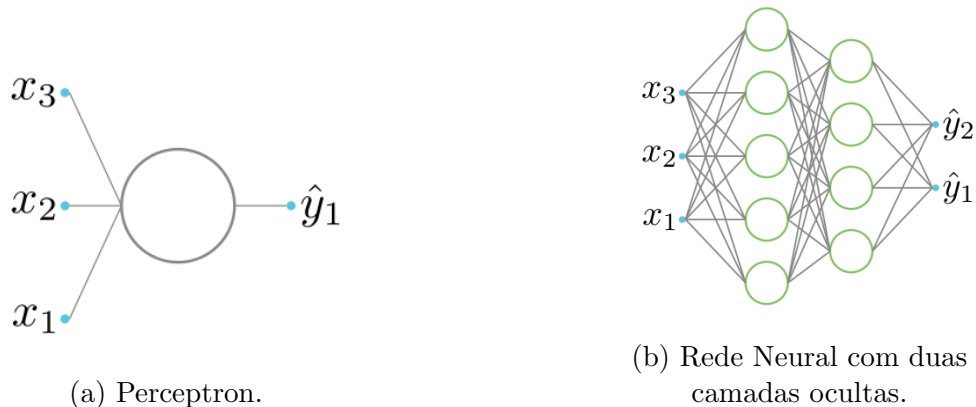


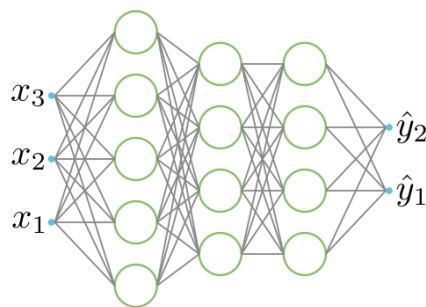
Figura 1.5: Estrutura básica de um neurônio artificial e uma rede neural com duas camadas ocultas.

1.2.1 Arquiteturas de Redes Neurais

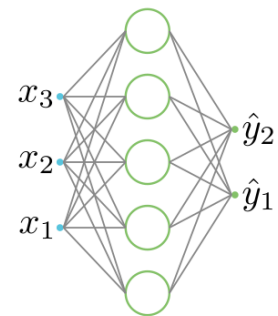
Existem diversas arquiteturas de redes neurais, cada uma projetada para lidar com diferentes tipos de problemas e dados. Algumas das arquiteturas mais comuns incluem:

- *Redes Neurais Feedforward (FNNs)*: São as redes neurais mais simples, onde a informação flui em uma única direção, da camada de entrada para a camada de saída, passando pelas camadas ocultas. Não há ciclos ou loops na rede. FNNs são amplamente utilizadas para tarefas de classificação e regressão.

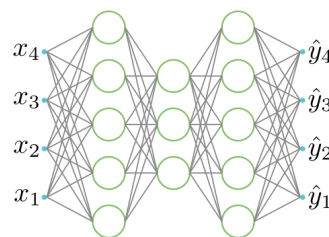
- *Redes Neurais Convolucionais (CNNs)*: São projetadas para processar dados com uma estrutura de grade, como imagens. Elas utilizam operações de convolução para extrair características espaciais dos dados de entrada. CNNs são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos e detecção de faces.
- *Redes Neurais Recorrentes (RNNs)*: São projetadas para lidar com dados sequenciais, onde a saída em um determinado momento depende das entradas anteriores. RNNs possuem conexões recorrentes que permitem que informações sejam mantidas ao longo do tempo. Elas são amplamente utilizadas em tarefas como processamento de linguagem natural e reconhecimento de fala.
- *Redes Neurais Generativas Adversariais (GANs)*: Consistem em duas redes neurais que competem entre si: um gerador e um discriminador. O gerador cria dados falsos, enquanto o discriminador tenta distinguir entre dados reais e falsos. GANs são amplamente utilizadas para geração de imagens realistas e outras tarefas criativas.
- *Encoder-Decoder*: São arquiteturas que consistem em duas partes principais: o encoder, que comprime os dados de entrada em uma representação latente, e o decoder, que reconstrói os dados a partir dessa representação. Essas redes são amplamente utilizadas em tarefas como tradução automática e geração de texto.



(a) Redes Neurais Feedforward (FNNs).



(b) Redes Neurais Recorrentes (RNNs).



(c) Encoder-Decoder.

Figura 1.6: Arquiteturas de Redes Neurais.

A Figura 1.6 ilustra algumas das arquiteturas de redes neurais mencionadas. Cada arquitetura possui suas próprias características e é adequada para diferentes tipos de problemas e dados. A escolha da arquitetura correta é crucial para o sucesso de um modelo de rede neural em uma tarefa específica.

1.2.2 Backpropagation

Assim como no perceptron, as redes neurais utilizam algoritmos de aprendizagem para ajustar seus pesos e vieses com o objetivo de minimizar uma função de custo. O algoritmo mais comum utilizado para esse propósito é o *backpropagation* (retropropagação) [4] e [12], que é uma extensão do método da descida do gradiente. O backpropagation é utilizado para calcular os gradientes da função de custo em relação aos pesos e vieses de todas as camadas da rede neural, permitindo a atualização eficiente desses parâmetros durante o treinamento. O algoritmo funciona propagando o erro da saída da rede de volta para as camadas anteriores, ajustando os pesos com base na contribuição de cada neurônio para o erro total.

Esse processo é repetido iterativamente até que a função de custo seja minimizada, resultando em uma rede neural treinada capaz de realizar previsões precisas em novos dados. Essa propagação do erro é feita utilizando a regra da cadeia do cálculo diferencial, que permite calcular as derivadas parciais de funções compostas. O algoritmo de backpropagation pode ser resumido nos seguintes passos:

Input: Dados de treinamento (x, y) , taxa de aprendizado η , pesos W e vieses b

Output: Pesos W e vieses b atualizados

repeat

Forward Pass: propagar x pela rede (camada a camada) e obter $\hat{y}$;

Cálculo do Erro: calcular o erro comparando $\hat{y}$ com y usando uma função de custo ;

Backward Pass: retropropagar o erro e calcular os gradientes $\nabla_W C$ e $\nabla_b C$ (regra da cadeia);

Atualização: atualizar pesos e vieses (descida do gradiente);

$W \leftarrow W - \eta \nabla_W C$;

$b \leftarrow b - \eta \nabla_b C$;

until critério de parada (ex.: n^o de épocas ou convergência);

Algorithm 2: Etapas do treinamento via Backpropagation

A Figura 1.7 ilustra o processo de backpropagation em uma rede neural:

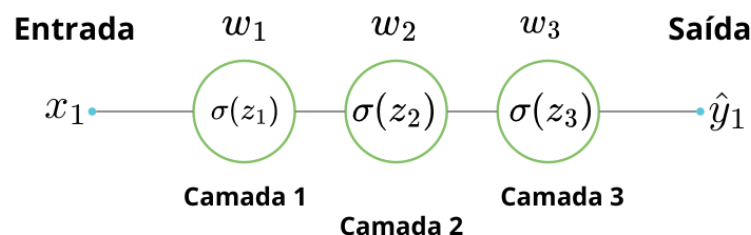


Figura 1.7: Ilustração do processo de backpropagation em uma rede neural.

Caso seja necessário ajustar o peso da primeira camada, o erro é propagado de volta da saída para a entrada, ajustando os pesos de cada camada com base na contribuição de cada neurônio para o erro total. Então:

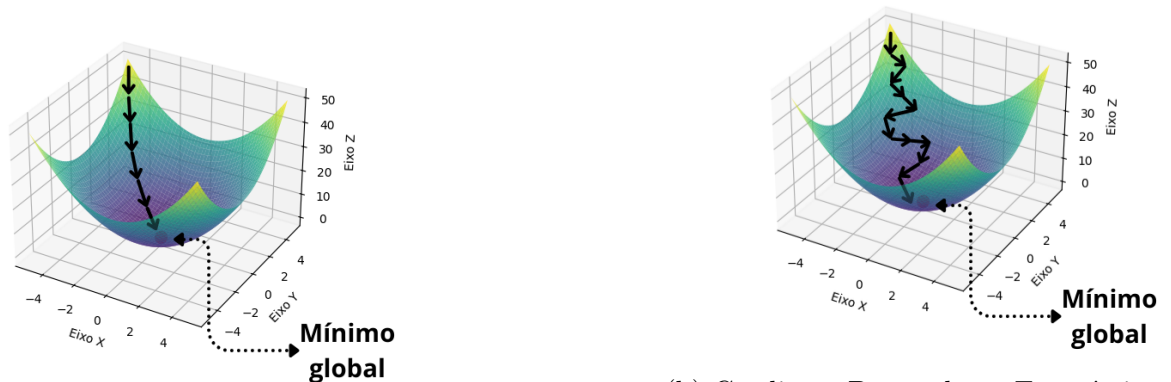
$$\nabla_w C = \frac{\partial C(w, b)}{\partial w_1} = \frac{C(w, b)}{\sigma(z_3)} \cdot \frac{\sigma(z_3)}{z_3} \cdot \frac{z_3}{\sigma(z_2)} \cdot \frac{\sigma(z_2)}{z_2} \cdot \frac{z_2}{\sigma(z_1)} \cdot \frac{\sigma(z_1)}{z_1} \cdot \frac{z_1}{w_1}, \quad (1.6)$$

em que z_1 , z_2 e z_3 são as saídas das camadas intermediárias e $C(w, b)$ é a função de custo do Erro quadrático médio, ou seja, $C(w, b) = \frac{(\hat{y} - \sigma(z_3))^2}{3}$.

1.2.3 Gradiente Descendente Estocástico

O Gradiente Descendente Estocástico (SGD - Stochastic Gradient Descent) [7] é uma variação do método de descida do gradiente tradicional, amplamente utilizado no treinamento de redes neurais e outros modelos de aprendizado de máquina. A principal diferença entre o SGD e o método de descida do gradiente padrão é a forma como os gradientes são calculados e atualizados durante o treinamento. No método de descida do gradiente tradicional, os gradientes são calculados usando todo o conjunto de dados de treinamento, o que pode ser computacionalmente caro e demorado, especialmente para grandes conjuntos de dados. Em contraste, o SGD calcula os gradientes usando apenas uma amostra aleatória (ou um pequeno lote) do conjunto de dados em cada iteração. Isso torna o processo de atualização dos pesos mais rápido e eficiente, permitindo que o modelo aprenda mais rapidamente.

A Figura 1.8 ilustra a diferença entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico, em que cada seta preta indica uma atualização dos pesos do modelo:



(a) Descida do Gradiente Tradicional.

(b) Gradiente Descendente Estocástico (SGD).

Figura 1.8: Comparação entre o método de descida do gradiente tradicional e o Gradiente Descendente Estocástico.

Apesar da Descida do Gradiente padrão fornecer uma direção mais precisa para o mínimo global da função de custo, o SGD é capaz de chegar a esse mínimo de forma mais rápida e eficiente, especialmente em grandes conjuntos de dados. Além disso, o SGD pode ajudar a escapar de mínimos locais devido à sua natureza estocástica, o que pode ser benéfico para o treinamento de redes neurais profundas.

1.3 Overfitting

O Overfitting [3] (do inglês "Sobreajuste" ou "Ajuste Excessivo") é um problema comum em aprendizado de máquina, onde um modelo aprende os detalhes e o ruído dos dados de treinamento a tal ponto que ele se torna incapaz de generalizar para novos dados. Isso significa

que o modelo tem um desempenho excelente nos dados de treinamento, mas falha ao fazer previsões precisas em dados não vistos anteriormente. O overfitting ocorre quando o modelo é muito complexo em relação à quantidade e qualidade dos dados disponíveis para treinamento. Modelos com muitos parâmetros, como redes neurais profundas, são particularmente suscetíveis ao overfitting, especialmente quando o conjunto de dados de treinamento é pequeno ou contém ruído significativo.

1.3.1 Hyperparâmetros

Um algoritmo de aprendizado pode ser visto como uma função que recebe dados de treinamento como entrada e produz como saída uma função ou modelo (i.e um conjunto de funções). No entanto, na prática muitos algoritmos de aprendizado possuem parâmetros adicionais que não são aprendidos diretamente a partir dos dados de treinamento, mas que influenciam o processo de aprendizado. Esses parâmetros são chamados de *hiperparâmetros* [2] e sua escolha adequada é crucial para o desempenho do modelo. Alguns exemplos comuns de hiperparâmetros incluem:

- *Taxa de Aprendizado*: Controla o tamanho dos passos dados durante a atualização dos pesos do modelo. Uma taxa de aprendizado muito alta pode levar a oscilações e impedir a convergência, enquanto uma taxa muito baixa pode resultar em um processo de treinamento lento. Valores típicos para a taxa de aprendizado variam entre 0.001 e 0.1, dependendo do problema e do algoritmo utilizado.
- *Número de Épocas*: Define quantas vezes o algoritmo de aprendizado passará por todo o conjunto de dados de treinamento. Um número muito alto pode levar ao overfitting, enquanto um número muito baixo pode resultar em underfitting (subajuste). Uma técnica comum para determinar o número ideal de épocas é o uso do *Early Stopping* (do inglês Parada Antecipada), que monitora o desempenho do modelo em um conjunto de validação e interrompe o treinamento quando o desempenho começa a piorar.
- *Tamanho do Lote (Batch Size)*: Refere-se ao número de amostras de dados usadas para calcular o gradiente em cada iteração do treinamento. Tamanhos de lote menores podem levar a atualizações mais ruidosas, enquanto tamanhos maiores podem resultar em um treinamento mais estável, mas mais lento. Os valores escolhidos geralmente variam entre 1 e 32, sendo que valores acima de 10 tiram proveito da Unidade Central de Processamento - CPU e da Unidade de Processamento Gráfico - GPU, que oferece ganho de velocidade dos produtos matriz-matriz em relação aos produtos matriz-vetor.
- *Arquitetura do Modelo*: Inclui o número de camadas, o número de neurônios em cada camada e o tipo de funções de ativação utilizadas. Modelos mais complexos podem capturar padrões mais sofisticados, mas também são mais propensos ao overfitting.

1.3.2 Regularização

A regularização é uma técnica utilizada para prevenir o overfitting em modelos de aprendizado de máquina, adicionando uma penalidade à função de custo que o modelo tenta minimizar durante o treinamento. Essa penalidade incentiva o modelo a manter os pesos dos parâmetros pequenos, o que ajuda a evitar que ele se ajuste excessivamente aos dados de treinamento. Existem várias técnicas de regularização, sendo as mais comuns:

- *Regularização L1 (Lasso)*: Adiciona uma penalidade proporcional à soma dos valores absolutos dos pesos dos parâmetros à função de custo. Isso pode levar a soluções esparsas,

onde alguns pesos são reduzidos a zero, efetivamente removendo certas características do modelo. A equação da função de custo com regularização L1 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i |w_i|, \quad (1.7)$$

em que $C_0(w, b)$ é a função de custo original, λ é o parâmetro de regularização que controla a força da penalidade, e w_i são os pesos dos parâmetros do modelo e n é o número de amostras no conjunto de dados.

- *Regularização L2 (Ridge)*: Adiciona uma penalidade proporcional à soma dos quadrados dos pesos dos parâmetros à função de custo. Isso incentiva o modelo a distribuir os pesos de forma mais uniforme, evitando que alguns pesos se tornem muito grandes. A equação da função de custo com regularização L2 é dada por:

$$C(w, b) = C_0(w, b) + \frac{\lambda}{n} \sum_i w_i^2, \quad (1.8)$$

em que os termos são definidos da mesma forma que na regularização L1.

- *Dropout*: Durante o treinamento, essa técnica desativa aleatoriamente uma fração dos neurônios em cada camada da rede neural. Isso força a rede a aprender representações mais robustas e reduz a dependência de neurônios específicos, ajudando a prevenir o overfitting. Em termos gerais, ao aplicar o dropout estamos treinando uma coleção de sub-redes dentro da rede original, o que promove a generalização do modelo.

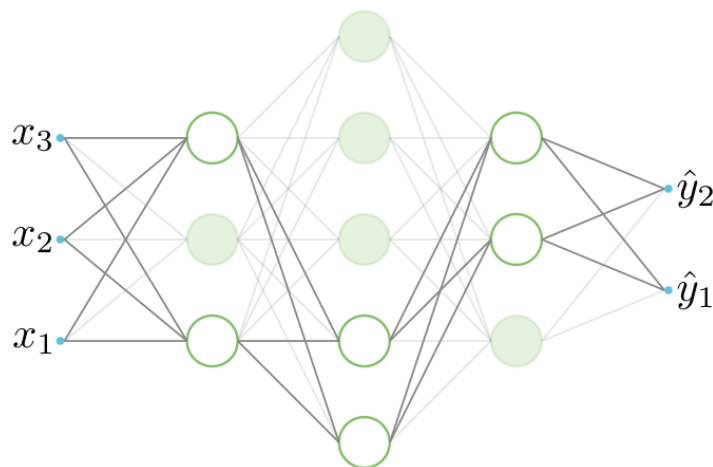


Figura 1.9: Ilustração do Dropout removendo 60% dos neurônios aleatoriamente durante uma época de treinamento.

- *Early Stopping*: Como Monitora o desempenho do modelo em um conjunto de validação durante o treinamento e interrompe o processo quando o desempenho começa a piorar. Isso evita que o modelo continue a se ajustar aos dados de treinamento além do ponto ideal. Como ilustrado na Figura 1.10, o ponto de parada ideal é onde a acurácia na validação começa a diminuir, indicando o início do overfitting.

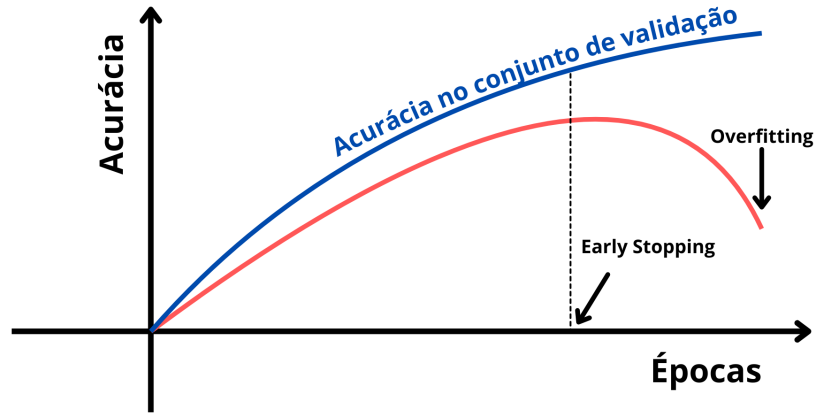


Figura 1.10: Ilustração do Early Stopping.

1.3.3 Momentum

O momentum [9] é uma técnica utilizada para acelerar o processo de treinamento de redes neurais, especialmente em situações onde a função de custo apresenta regiões íngremes ou planas. A ideia principal do momentum é acumular uma média ponderada dos gradientes anteriores para suavizar as atualizações dos pesos, permitindo que o modelo mantenha uma direção consistente durante o treinamento. Isso ajuda a evitar oscilações e acelera a convergência para o mínimo da função de custo. Dada uma função de custo $C(w, b)$ a ser minimizada, o momentum clássico é definido por:

$$\begin{cases} v_{t+1} = \beta v_t - 1 + \epsilon \nabla C(w_t, b_t) \\ \theta_{t+1} = \theta_t + v_{t+1}, \end{cases} \quad (1.9)$$

em que $\epsilon > 0$ é a taxa de aprendizagem, $\beta \in [0, 1]$ é o coeficiente de momentum, v_t é a velocidade acumulada no tempo t , $\theta_t = (w_t, b_t)$ representa os parâmetros do modelo (pesos e vieses) no tempo t e $\nabla C(w_t, b_t)$ é o gradiente da função de custo em relação aos parâmetros do modelo. O coeficiente de momentum β controla a influência dos gradientes anteriores nas atualizações atuais. Valores típicos para β variam entre 0.9 e 0.99, com valores mais altos dando mais peso aos gradientes passados.

1.3.4 Vanishing Gradient

O problema do *vanishing gradient* (gradiente que desaparece) ocorre durante o treinamento de redes neurais profundas, onde os gradientes calculados durante o processo de backpropagation tornam-se extremamente pequenos à medida que são propagados para as camadas mais próximas da entrada. Isso acontece devido à multiplicação repetida de pequenas derivadas associadas às funções de ativação e aos pesos das camadas. Como resultado, os pesos das camadas iniciais da rede recebem atualizações insignificantes, dificultando o aprendizado efetivo dessas camadas.

Exemplo 1.2 Considere uma rede neural profunda com várias camadas ocultas, onde cada camada utiliza a função de ativação sigmoid logística, definida como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.10)$$

A derivada dessa função é dada por:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (1.11)$$

Durante o processo de *backpropagation*, o gradiente da função de custo em relação aos pesos das camadas iniciais é calculado utilizando a regra da cadeia. Isso envolve a multiplicação das derivadas das funções de ativação de cada camada, como mostra 1.6. Como a derivada da função sigmoid é sempre menor que 0.25 (o valor máximo ocorre em $z = 0$), a multiplicação repetida dessas pequenas derivadas resulta em gradientes que se tornam cada vez menores à medida que são propagados para trás através das camadas da rede. Por exemplo, se uma rede possui 10 camadas ocultas, o gradiente pode ser reduzido a um valor extremamente pequeno, como 10^{-6} ou menor, tornando as atualizações dos pesos nas camadas iniciais praticamente insignificantes.

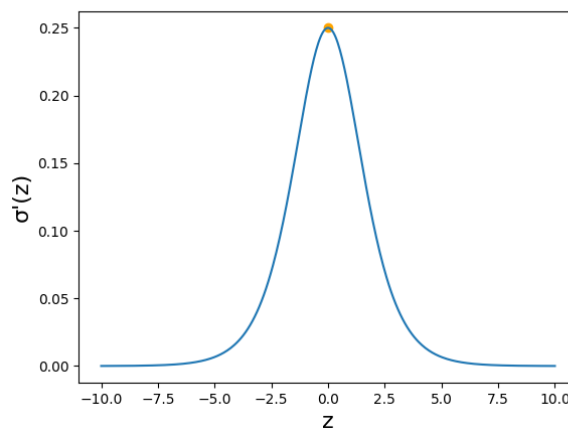


Figura 1.11: Gráfico da derivada da função sigmoid logística.

O problema do vanishing gradient é particularmente comum em redes neurais recorrentes (RNNs) e em redes feedforward profundas que utilizam funções de ativação como a sigmoid ou a tangente hiperbólica. Para contornar o problema do vanishing gradient, várias técnicas podem ser empregadas, incluindo:

- *Uso de Funções de Ativação Alternativas:* Funções de ativação como ReLU (Rectified Linear Unit) e suas variantes (Leaky ReLU, Parametric ReLU) têm derivadas constantes para valores positivos, o que ajuda a manter os gradientes maiores durante o *backpropagation*.
- *Batch Normalization:* Normaliza as ativações de cada camada durante o treinamento, o que pode ajudar a estabilizar os gradientes e acelerar o processo de aprendizado.

1.4 Redes Neurais Convolucionais

As redes neurais convolucionais (RNCs) são uma classe especial de redes neurais projetadas para processar dados com uma estrutura de grade e que tenham uma relação de vizinhança entre os elementos, como imagens [10]. Elas são amplamente utilizadas em tarefas de visão computacional, como reconhecimento de objetos, detecção de faces e segmentação de imagens. As RNCs são inspiradas na organização do córtex visual dos animais, onde diferentes regiões do cérebro são responsáveis por processar diferentes partes do campo visual.

1.4.1 Cotex Visual Humano

O córtex visual humano é a parte do cérebro responsável pelo processamento das informações visuais recebidas pelos olhos [8]. Ele está localizado na parte posterior do cérebro, na região occipital. O córtex visual é organizado em várias áreas, cada uma especializada em processar diferentes aspectos da visão, como cor, forma, movimento e profundidade. Essas áreas trabalham em conjunto para interpretar e compreender o mundo visual ao nosso redor. A estrutura do córtex visual é altamente complexa, com bilhões de neurônios interconectados que formam redes neurais intrincadas. Essas redes são capazes de aprender e adaptar-se com base nas experiências visuais, permitindo-nos reconhecer objetos, rostos e padrões visuais de maneira eficiente. Uma característica importante do córtex visual é a presença de campos receptivos, que são regiões específicas do campo visual onde a estimulação de um neurônio resulta em uma resposta. Esses campos receptivos são organizados de forma hierárquica, com neurônios em níveis mais baixos respondendo a características simples, como bordas e texturas, enquanto neurônios em níveis mais altos respondem a características mais complexas, como formas e objetos completos.

1.4.2 Convolução

A operação de convolução é uma operação matemática linear entre duas funções. Sejam $x(t)$ e $w(a)$ funções reais contínuas e integráveis, respectivamente denominadas de *senal* e *kernel*, em que t pode ser representado pela variável **tempo**, a convolução entre essas duas funções é definida como:

$$s(t) = (x * w)(t) = \int_{-\infty}^{\infty} x(a)w(t - a)da, \quad (1.12)$$

em que $*$ denota a operação de convolução. No contexto de redes neurais convolucionais, a convolução é aplicada a dados discretos, como imagens representadas por matrizes de pixels. Nesse caso, o sinal x é uma matriz bidimensional que representa a imagem de entrada e o *kernel* w é uma matriz menor que atua como um filtro para extrair características específicas da imagem. A operação de convolução em dados discretos é realizada deslizando o *kernel* sobre a imagem e calculando a soma ponderada dos valores dos pixels cobertos pelo *kernel*. A saída da convolução é uma nova matriz, chamada de mapa de características, que destaca as características extraídas pela aplicação do *kernel*.

No caso de uma convolução bidimensional, a operação é definida como:

$$S(i, j) = (X * W)(i, j) = \sum_m \sum_n X(m, n)W(i - m, j - n), \quad (1.13)$$

considerando que X é a matriz da imagem de entrada, W é a matriz do kernel e S é o mapa de características resultante.

melhorar a figura/ deixar texto mais visível

Na Figura 1.12 é ilustrado um exemplo de convolução bidimensional, onde uma matriz de entrada X é convoluída com um *kernel* W para produzir a saída S . O *kernel* é deslizado sobre a matriz de entrada, e em cada posição, a soma ponderada dos valores dos pixels cobertos pelo *kernel* é calculada para gerar o valor correspondente na matriz de saída. O cálculo da largura e da altura da imagem de saída pode ser feito utilizando as seguintes fórmulas:

$$w_{out} = w_{in} - w_{kernel} + 1, \quad (1.14)$$

$$h_{out} = h_{in} - h_{kernel} + 1. \quad (1.15)$$

Em que w_{out} e h_{out} são a largura e a altura da imagem de saída, w_{in} e h_{in} são a largura e a altura da imagem de entrada, e w_{kernel} e h_{kernel} são a largura e a altura do *kernel*, respectivamente. A Figura 1.13 mostra diferentes aplicações de *kernels* em uma imagem.

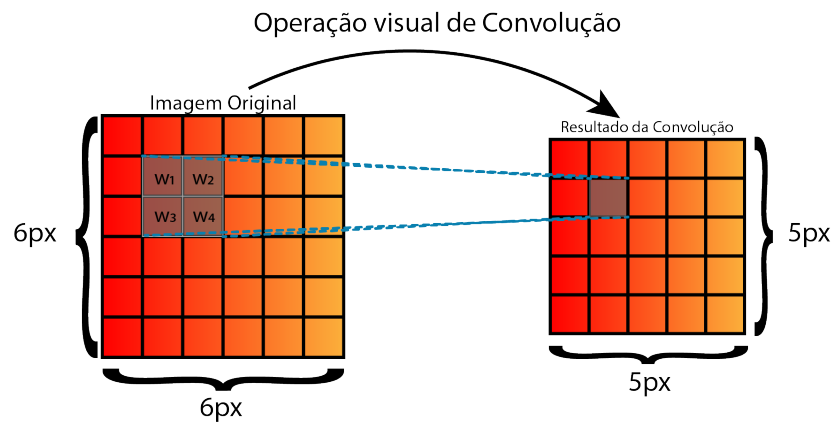


Figura 1.12: Exemplo de convolução bidimensional. A função $X(t)$ é convoluída com o *kernel* $W(a)$ para produzir a saída $S(t)$.

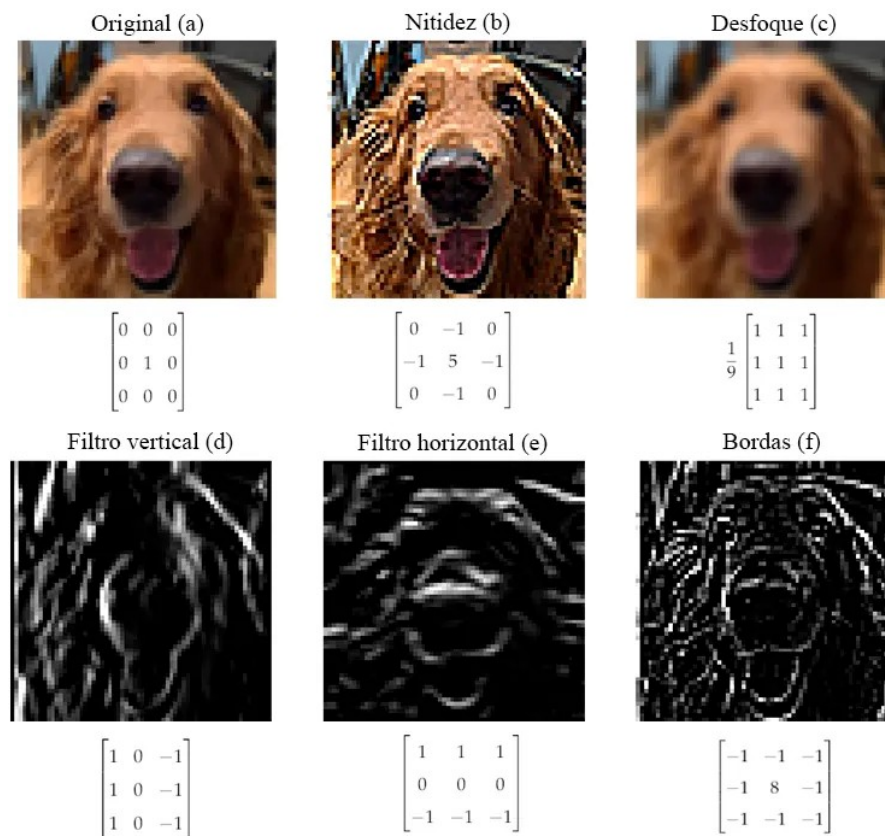


Figura 1.13: Aplicações de diferentes *kernels* em uma imagem. A imagem original é representada em (a), juntamente com o *kernel* identidade, que não faz modificações. Em (b) a imagem passou por um *kernel* que aumenta a nitidez reforçando o pixel central e aumentando a diferença dele em relação à sua vizinhança. Em (c) foi usado um *kernel* de desfoque, que substitui um pixel pela média dele com seus vizinhos. A imagem em (d) é o resultado de aplicar na imagem em escala de cinza um *kernel* que reforça suas linhas verticais, enquanto (e) reforça suas linhas horizontais. O *kernel* em (f) reforça os contornos (bordas) da imagem.

Normalmente os *kernels* são muito menores do que as imagens, e através da operação de convolução um *kernel* é utilizado para processar toda uma imagem, portanto, diz-se que os

parâmetros são compartilhados. Por conta dessa representação reduzida, cada camada da rede convolucional deve aprender uma quantidade menor de parâmetros.

Em CNNs, normalmente consideramos a convolução como uma camada da rede, e cada camada normalmente possui múltiplos *kernels*, produzindo a mesma quantidade de mapas de atributos, cada um responsável por extrair diferentes características da imagem de entrada. Durante o treinamento da rede, os valores dos *kernels* são ajustados para otimizar a capacidade da rede de reconhecer padrões específicos nas imagens. Por exemplo, uma camada convolucional com 3 *kernels* que recebe como entrada uma imagem em escala de cinza de dimensões $(w_{in} \times h_{in} \times 1)$ produzirá uma saída com dimensões $(w_{out} \times h_{out} \times 3)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em 1.14 e 1.15. A saída não é mais uma imagem comum, mas sim um conjunto de três mapas de características, cada um representando a resposta da imagem de entrada a um dos três *kernels* aplicados. Encadeando os mapas de atributos, a rede pode aprender representações hierárquicas das características presentes nas imagens, desde bordas e texturas simples até formas e objetos mais complexos.

1.4.3 *Kernel* Multicamadas

Em uma imagem com mais de um canal, como no caso de uma imagem RGB (vermelho, verde, azul) do inglês (*Red, Green, Blue*), a convolução acontece com um *kernel* em cada canal da imagem, um conjunto de canais é chamado de filtro. Por exemplo, considere uma imagem RGB de dimensões $(w_{in} \times h_{in} \times 3)$ e um *kernel* de dimensões $(k_w \times k_h \times 3)$. A convolução é realizada separadamente em cada canal (vermelho, verde e azul) da imagem, utilizando o respectivo canal do *kernel*. Após a aplicação do *kernel* em cada canal, os resultados são somados ou é aplicada uma média para produzir um único mapa de características. Portanto, a saída da convolução terá dimensões $(w_{out} \times h_{out} \times 1)$, onde w_{out} e h_{out} são calculados conforme as fórmulas apresentadas em 1.14 e 1.15.

1.4.4 Padding

Durante a aplicação da convolução, é comum que a dimensão espacial (largura e altura) da imagem de saída seja menor do que a da imagem de entrada. Isso ocorre porque o *kernel* não pode ser aplicado nas bordas da imagem sem ultrapassar seus limites. Para mitigar essa redução de dimensão, uma técnica chamada *padding* é frequentemente utilizada. O padding envolve adicionar uma borda de pixels ao redor da imagem original antes de aplicar a convolução. Esses pixels adicionais podem ser preenchidos com zeros (*zero-padding*) ou com valores replicados dos pixels mais próximos da borda da imagem original, (*reflection padding*). O uso de padding permite que o *kernel* seja aplicado em todas as regiões da imagem, incluindo as bordas, preservando assim as dimensões espaciais da imagem de entrada na saída.

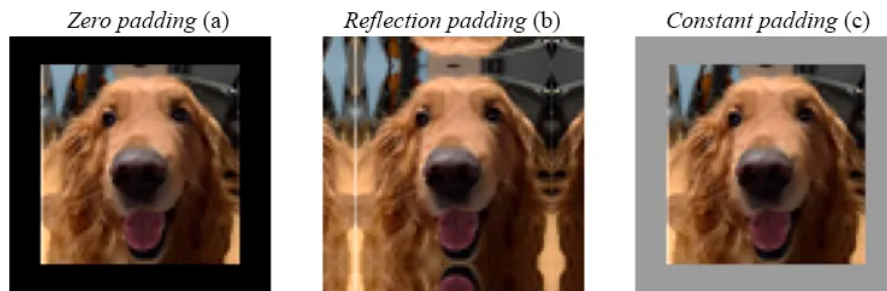


Figura 1.14: Aplicação de padding em uma imagem. Em (a) a imagem original passou por zero padding. Em (b) a imagem original passou por *reflection padding*. A imagem em (c) passou por um padding constante, semelhante em (a), mas com valor diferente de zero.

O cálculo da largura e da altura da imagem de saída com padding pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= w_{in} - w_{kernel} + 2w_{pad} + 1, \\ h_{out} &= h_{in} - h_{kernel} + 2h_{pad} + 1, \end{aligned} \quad (1.16)$$

onde w_{pad} e h_{pad} são a quantidade de pixels adicionados como padding na largura e na altura, respectivamente.

1.4.5 Pooling

O pooling é uma operação utilizada em redes neurais convolucionais para reduzir as dimensões espaciais (largura e altura) dos mapas de características, mantendo as informações mais relevantes. Essa redução ajuda a diminuir a complexidade computacional da rede, além de tornar o modelo mais robusto a pequenas variações na entrada, como translações e rotações. Existem diferentes tipos de operações de pooling, sendo as mais comuns o *Max Pooling* e o *Average Pooling*. A técnica de pooling substitui agrupamentos de pixels por um valor que os represente. A figura 1.15 ilustra o funcionamento do max pooling de tamanho 2, em que os pixels são agrupados em quadrados de tamanho 2x2 e são substituídos pelo maior valor de intensidade de pixel do grupo.

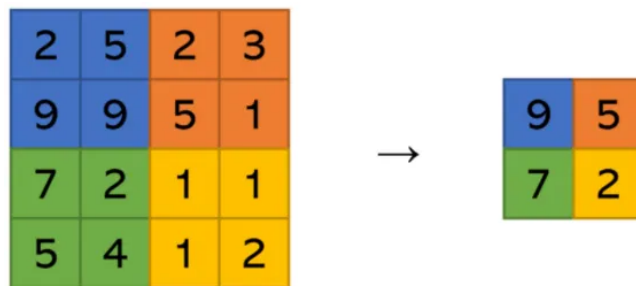


Figura 1.15: Exemplo de Max Pooling com tamanho de 2x2. A imagem original é reduzida para a imagem menor após a aplicação do Max Pooling. Cada bloco de 2x2 pixels na imagem original é substituído pelo valor máximo dentro desse bloco na imagem reduzida.

Como consequência imediata a aplicação do *pooling*, a informação fica mais condensada, o que pode levar a uma perda de detalhes finos na imagem. No entanto, essa perda é compensada pela capacidade do modelo de focar nas características mais importantes e gerais da imagem, melhorando sua capacidade de generalização em tarefas de reconhecimento de padrões.

1.4.6 Stride

O *stride* (passo) é um parâmetro utilizado nas operações de convolução e pooling em redes neurais convolucionais que determina o número de pixels que o *kernel* de pooling se desloca ao longo da imagem de entrada. Em outras palavras, o stride define o quanto o *kernel* avance a cada aplicação da operação. Um stride de 1 significa que o *kernel* se mova um pixel por vez, enquanto um stride maior que 1 faz com que o *kernel* pule pixels, resultando em uma redução mais rápida das dimensões espaciais da imagem. O uso do stride afeta diretamente o tamanho da saída gerada pela operação de convolução ou pooling. O cálculo da largura e da altura da imagem de saída com stride pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= \frac{w_{in} - w_{kernel} + 2w_{pad}}{s_w} + 1, \\ h_{out} &= \frac{h_{in} - h_{kernel} + 2h_{pad}}{s_h} + 1, \end{aligned} \quad (1.17)$$

onde s_w e s_h são os valores do stride na largura e na altura, respectivamente.

1.4.7 Convolução Transposta

A convolução transposta, também conhecida como deconvolução ou convolução fracionária, é uma operação utilizada em redes neurais convolucionais para aumentar as dimensões espaciais (largura e altura) dos mapas de características. Essa operação é frequentemente empregada em tarefas de geração de imagens, onde o objetivo é reconstruir imagens a partir de representações compactas. A convolução transposta funciona de maneira inversa à convolução tradicional. Enquanto a convolução reduz as dimensões espaciais da imagem de entrada, a convolução transposta expande essas dimensões. Isso é alcançado aplicando o *kernel* de convolução de forma que ele “espalhe” os valores dos pixels da imagem de entrada para posições mais amplas na imagem de saída. A Figura 1.16 ilustra o processo pelo qual uma imagem 3×3 passa por uma convolução transposta de *kernel* 3×3 e stride 1 para gerar uma imagem 5×5 .

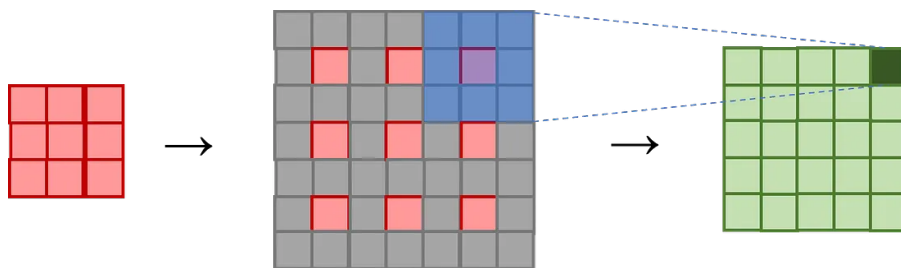


Figura 1.16: Convolução transposta. Uma convolução de *kernel* 3×3 com stride 1 em uma imagem 5×5 geraria uma imagem de dimensões 3×3 . A convolução transposta aplicada na imagem 3×3 (vermelha) para se obter uma imagem 5×5 (verde) é feita em duas etapas.

Primeiramente (centro) se insere pixels de valor 0, representados pela cor cinza, entre os pixels da imagem original, juntamente com pixels de borda. Em seguida se faz uma convolução nessa imagem alterada também usando um *kernel* 3×3 e stride 1.

O cálculo da largura e da altura da imagem de saída com convolução transposta pode ser ajustado conforme as seguintes fórmulas:

$$\begin{aligned} w_{out} &= (w_{in} - 1) \cdot s_w - 2w_{pad} + w_{kernel}, \\ h_{out} &= (h_{in} - 1) \cdot s_h - 2h_{pad} + h_{kernel}, \end{aligned} \quad (1.18)$$

onde s_w e s_h são os valores do stride na largura e na altura, respectivamente.

Exemplo 1.3 A figura 1.17 apresenta uma arquitetura típica de uma Rede Neural Convolucional (CNN).

1.5 U-net

Modificando a arquitetura tradicional de uma CNN, é possível obter variadas redes neurais com objetivos específicos. O uso típico de redes convolucionais ocorre em tarefas de classificação, onde o objetivo é atribuir uma etiqueta a uma imagem inteira. No entanto, em muitas tarefas visuais, especialmente no processamento de imagens biomédicas, a saída desejada deve incluir informações espaciais detalhadas, como a localização precisa de estruturas dentro da imagem. Para atender a essa necessidade, foi proposta a arquitetura U-Net [5], que é uma rede neural convolucional projetada para segmentação de imagens. A U-Net é composta por uma parte de

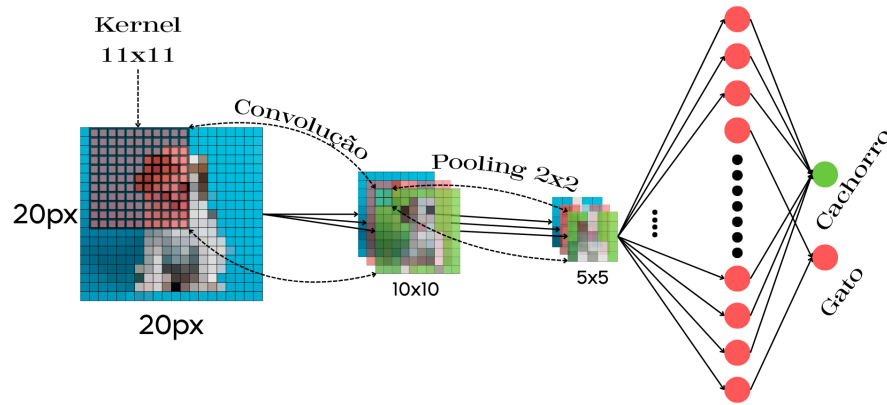


Figura 1.17: Arquitetura típica de uma Rede Neural Convolutional com uma camada de convolução e uma camada de pooling, em que é enviado para rede a imagem de um cachorro e é feita a classificação do objeto na imagem.

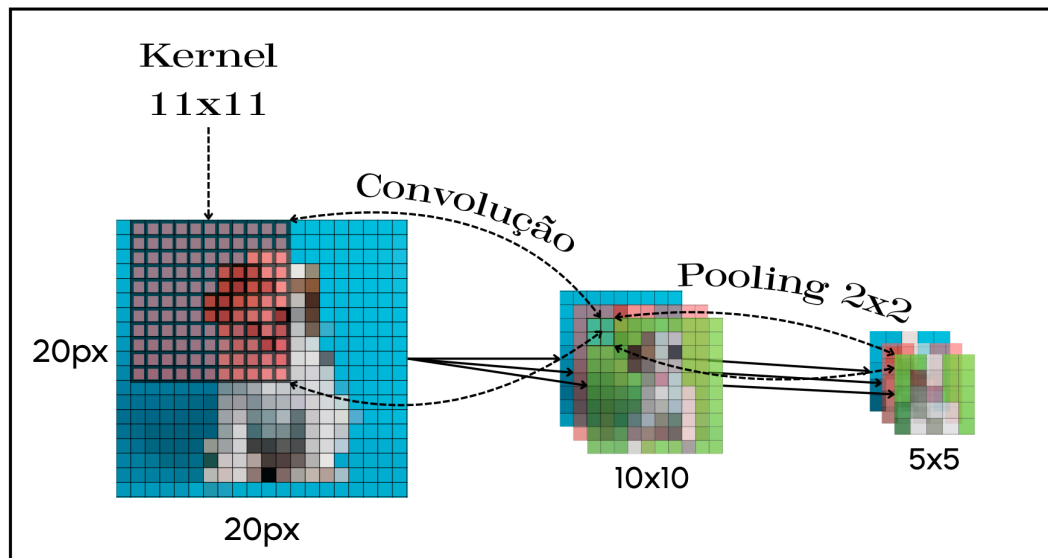
contração (encoder) e uma parte de expansão (decoder), formando uma estrutura em forma de U. O encoder é responsável por extrair características da imagem de entrada, enquanto o decoder reconstrói a imagem segmentada, utilizando as características extraídas pelo encoder. A arquitetura U-Net é especialmente eficaz em tarefas de segmentação semântica, onde o objetivo é classificar cada pixel da imagem em uma categoria específica, permitindo a identificação precisa de objetos e estruturas dentro da imagem.

Assim como no Exemplo 1.17, teríamos a parte de convolução e pooling para extrair as características da imagem e logo depois a parte de expansão para reconstruir a imagem segmentada, como podemos ver na Figura 1.18.

1.5.1 Copy and Crop

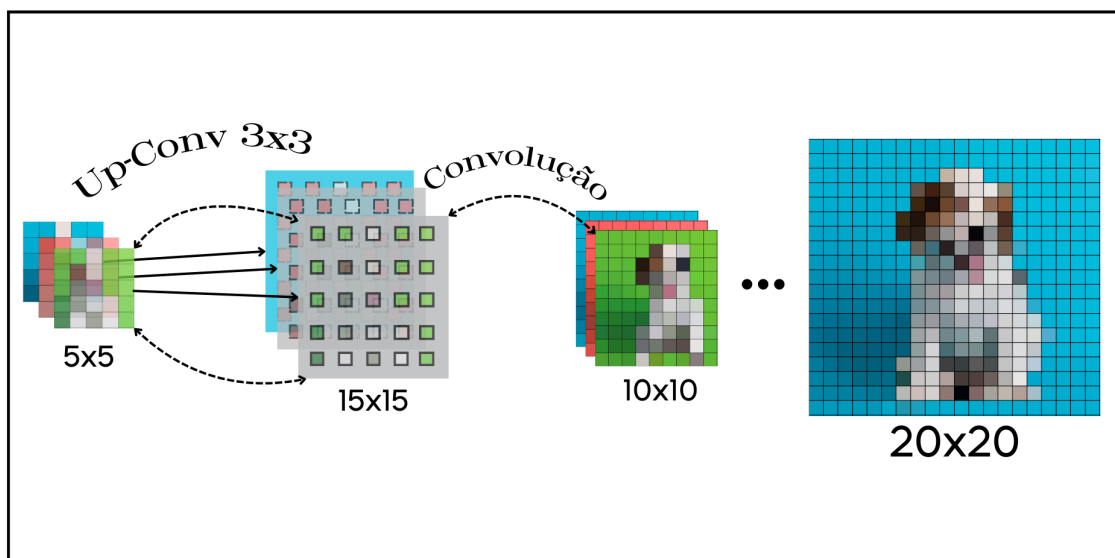
Uma característica importante da arquitetura U-Net é a presença de conexões de atalho (skip connections) entre as camadas correspondentes do encoder e do decoder. Essas conexões permitem que as informações de baixo nível, como bordas e texturas, sejam diretamente transferidas do encoder para o decoder, facilitando a reconstrução precisa da imagem segmentada. O processo de copiar e recortar (copy and crop) envolve copiar as características extraídas pelo encoder e recortá-las para se encaixar nas dimensões da camada correspondente no decoder. Isso é feito para garantir que as informações relevantes sejam preservadas durante a reconstrução da imagem segmentada, melhorando a precisão da segmentação, especialmente em regiões onde os detalhes são cruciais.

Bloco Encoder



(a) Encoder da U-Net, onde as setas indicam as operações de convolução e pooling que extraem características da imagem de entrada.

Bloco Decoder



(b) Decoder da U-Net que reconstrói a imagem segmentada utilizando as características extraídas pelo encoder.

Figura 1.18: Arquitetura da U-Net, composta por uma parte de contração (encoder) e uma parte de expansão (decoder). O encoder extrai características da imagem de entrada, enquanto o decoder reconstrói a imagem segmentada.

Referências Bibliográficas

- [1] Mark F. Bear, Barry W. Connors, and Michael A. Paradiso. *Neurociências: desvendando o sistema nervoso*. Artmed, Porto Alegre, 4 edition, 2017.
- [2] Yoshua Bengio. Practical recommendations for gradient-based training of deep architectures. *Cornell University*, (Version 2), 2012.
- [3] Deep Learning Book. Capítulo 19 – overfitting e regularização – parte 1, 2025. Acessado: 2026-01-26.
- [4] Robert Hecht-Nielsen. Theory of the backpropagation neural network. *Academic Press*, 1992.
- [5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241, Munich, Germany, 2015. Springer.
- [6] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 1958.
- [7] Sebastian Ruder. An overview of gradient descent optimization algorithms. *Insight Centre for Data Analytics, NUI Galway Aylien Ltd., Dublin*, 2016.
- [8] Cindy L. Stanfield. *Fisiologia Humana*. Pearson Education do Brasil, 2014.
- [9] Ilya Sutskever. On the importance of initialization and momentum in deep learning. *International Conference on Machine Learning*, 2013.
- [10] Vinícius Trevisan. Como funcionam as redes neurais convolucionais (cnns), 2021. Acessado: 2025-12-16.
- [11] Walter Pitts Warren S. McCulloch. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 1943.
- [12] Paul J. Werbos. Backpropagation through time: What it does and how to do it, 1990.